



UNIVERSIDAD
TECNOLÓGICA NACIONAL
FACULTAD REGIONAL
RESISTENCIA

TRABAJO FINAL INTEGRADOR

LABORATORIO DE PYTHON

Algoritmos y Estructuras de Datos

Integrantes:

- Bernardis, Ignacio Ezequiel
- De Bortoli, Juan Pablo
- López, María Josefina
- Morra Beneyto, Noel Victoria
- Veron, Malena Abril

ISI C
Comisión K1.3
Año 2026

INTRODUCCIÓN

La gestión de pedidos en restaurantes es una actividad fundamental para organizar el servicio, registrar ventas y brindar una mejor atención al cliente. Cuando estos procesos se realizan de forma manual, pueden aparecer errores en los pedidos, demoras, dificultades para calcular importes y problemas para llevar un control claro de la información.

Frente a esta situación, la digitalización permite automatizar tareas repetitivas y mejorar la organización del trabajo gastronómico. En este proyecto se desarrolla GoKitchen, una aplicación programada en Python que permite gestionar pedidos de distintos restaurantes, seleccionar productos, agregar toppings, calcular totales, aplicar descuentos y registrar las ventas realizadas.

De esta manera, el sistema busca ofrecer una herramienta simple y funcional que facilite la administración de pedidos y contribuya a un manejo más ordenado de la información dentro de un entorno gastronómico.

GoKitchen: Sistema de gestión de pedidos para restaurantes

GoKitchen es una aplicación desarrollada en Python mediante una interfaz por consola, diseñada para facilitar la gestión de pedidos en distintos restaurantes. El sistema permite seleccionar un restaurante, elegir productos del menú, personalizar los pedidos mediante toppings, calcular el importe total, aplicar descuentos cuando corresponda y registrar automáticamente cada venta en un archivo de texto. Además, genera reportes que permiten visualizar las ventas realizadas y obtener información útil para el control del negocio.

Problema que resolvemos

En muchos establecimientos gastronómicos, el registro de pedidos y ventas aún se realiza de forma manual, lo que puede generar errores en los cálculos, dificultades para organizar la información y demoras en la atención al cliente. Asimismo, la falta de un registro ordenado dificulta el seguimiento de las ventas y la obtención de reportes para analizar el funcionamiento del negocio.

Nuestro proyecto busca simplificar la gestión de pedidos mediante una herramienta digital que automatiza el registro de las ventas, reduce la posibilidad de errores y facilita la organización de la información de manera rápida y sencilla.

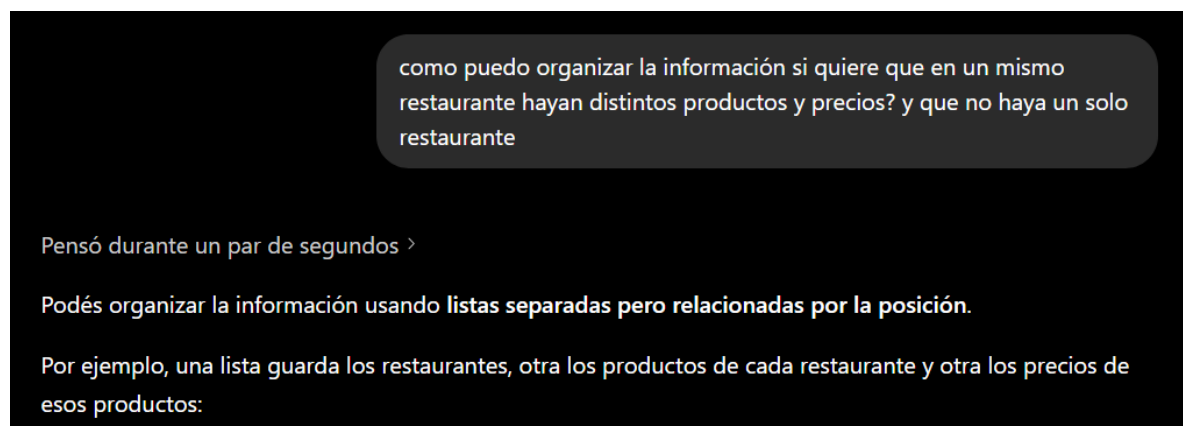
Solución propuesta

La aplicación presenta un menú por consola que permite:

- Registrar pedidos seleccionando un restaurante y los productos del menú.
- Personalizar los productos mediante toppings disponibles.
- Calcular automáticamente el importe total y aplicar descuentos cuando corresponda.
- Registrar cada venta en un archivo de texto con su fecha y hora.
- Generar reportes con información sobre las ventas realizadas, los medios de pago utilizados y el total vendido por restaurante.

DESARROLLO

APORTES DE LA INTELIGENCIA ARTIFICIAL



- Se muestra una consulta a la Inteligencia Artificial sobre cómo organizar la información del sistema cuando existen varios restaurantes, cada uno con distintos productos y precios. La respuesta propone usar listas separadas relacionadas por posición.

como puedo hacer que el usuario ingrese solamente números válidos en los menús y evitar que el programa se rompa?

Pensó durante un segundo >

Podés hacerlo con una función que repita la pregunta hasta que el usuario ingrese un número correcto.

Para evitar que el programa se rompa si escribe letras, se usa `try` y `except`. Y para evitar opciones fuera de rango, se valida con un `if`.

- Se consultó sobre validación de datos y organización interna del pedido. La IA explica cómo evitar errores cuando el usuario ingresa opciones inválidas y cómo guardar cada producto pedido con sus datos: nombre, precio, cantidad, toppings y precios.

que hago si quiero guardar cada producto pedido con varios datos juntos, por ejemplo nombre, precio, cantidad, toppings elegidos y precios de esos toppings

Pensó durante un segundo >

Podés guardar cada producto pedido en una **lista interna** con todos sus datos juntos, y después agregar esa lista al pedido general.

Por ejemplo:

 Python

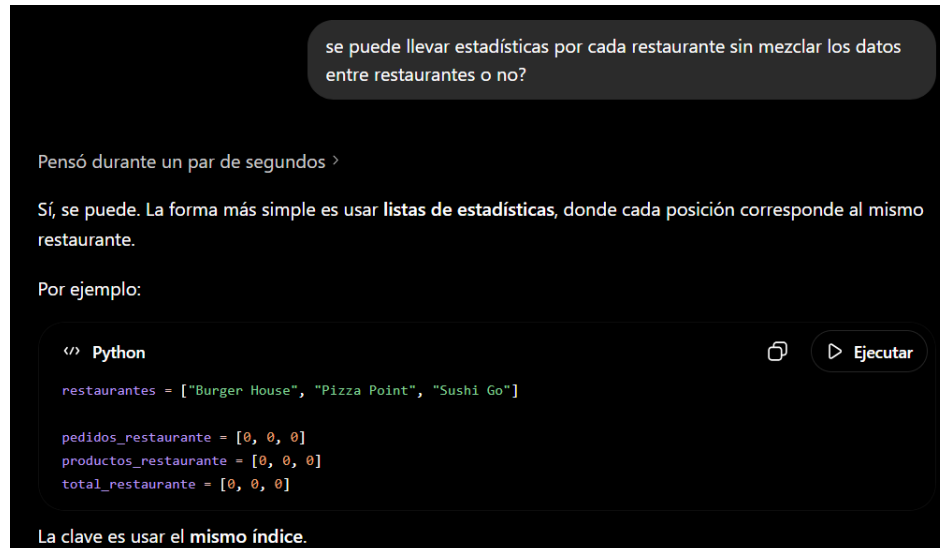


 Ejecutar

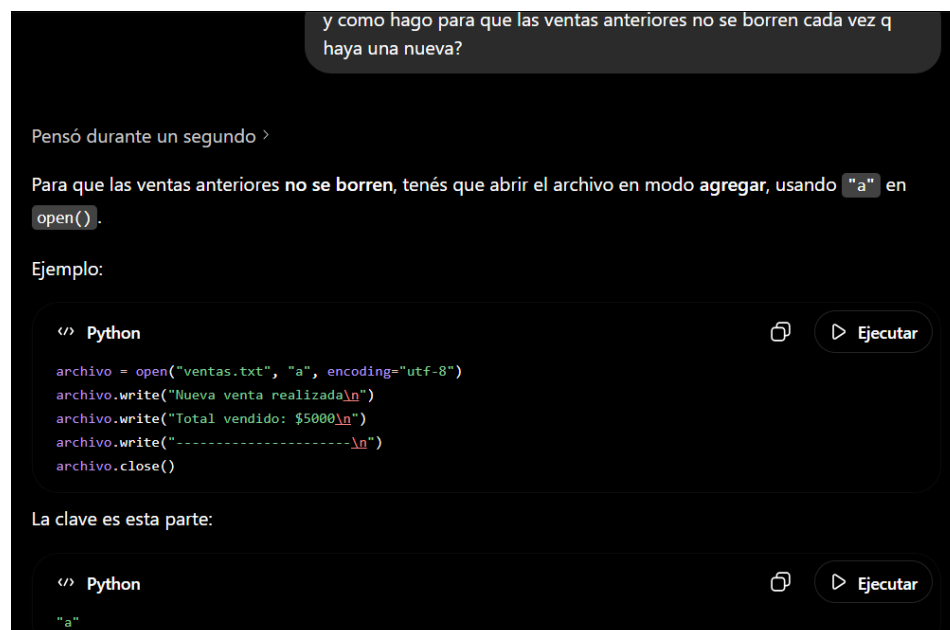
```
producto_pedido = [  
    nombre_producto,  
    precio_producto,  
    cantidad,  
    toppings_elegidos,  
    precios_toppings  
]  
  
pedido.append(producto_pedido)
```

- Aquí surgió la duda de cómo organizar internamente cada producto pedido dentro del sistema. La respuesta brindada por la Inteligencia Artificial no fue clave para continuar con el proyecto, nos explicó que se podía guardar cada producto como una lista

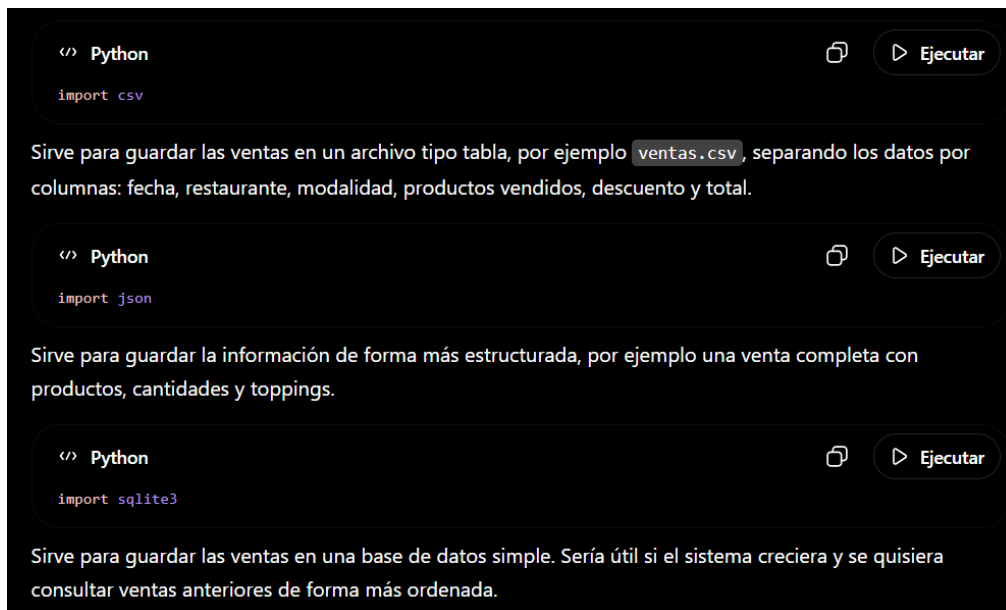
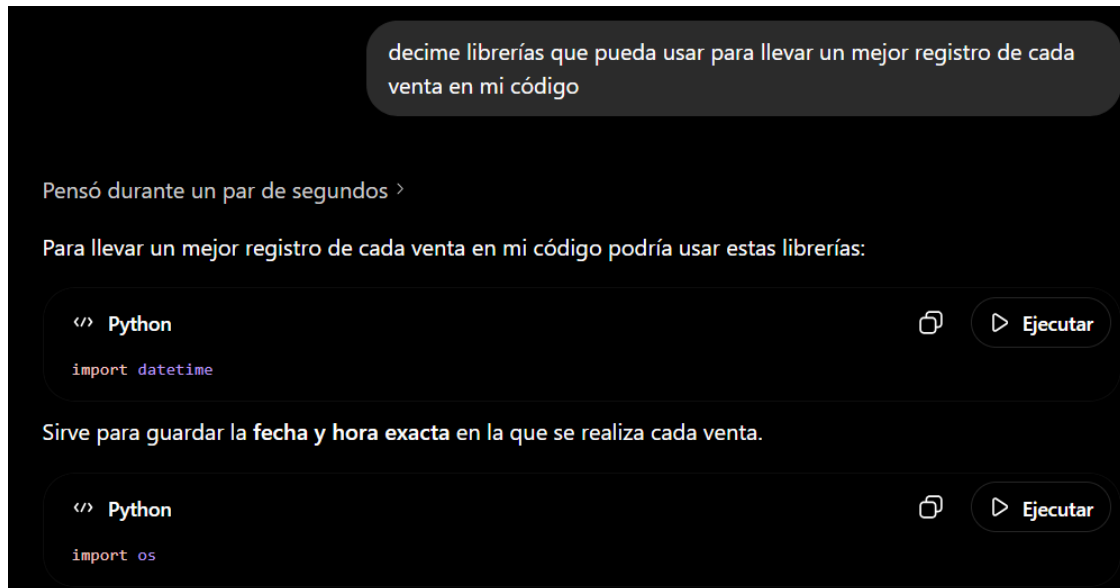
con sus datos principales: nombre, precio, cantidad, toppings elegidos y precios de esos toppings. Luego, esa lista se agrega al pedido general para poder calcular el total y mostrar el resumen final.



- En esta captura se preguntó cómo se debe registrar estadísticas por cada restaurante sin mezclar los datos. La IA resolvió la duda ya que nos dijo que se podían usar listas paralelas, donde cada posición corresponde al mismo restaurante. De esta manera, los pedidos realizados, productos vendidos y total vendido se actualizan usando el mismo índice del restaurante seleccionado.



- Se presenta una consulta sobre cómo evitar que las ventas anteriores se borren al registrar una nueva venta. La IA explica que se debe abrir el archivo en modo agregar, usando "a".



- Y, otra duda que surgió durante el desarrollo fue el uso de librerías de Python para mejorar el registro de las ventas. Se analizaron opciones como csv, json o sqlite3, que podrían servir para guardar la información de una manera más ordenada o avanzada. Sin embargo, en el código final se utilizaron solamente datetime y os: la primera para registrar la fecha y hora de cada venta, y la segunda para verificar si el archivo ventas.txt existía

antes de escribir en él. Estas dos librerías fueron suficientes para las necesidades del sistema desarrollado.

FUNCIONAMIENTO DEL CODIGO

Consola:

```
*****
TICKET - GO KITCHEN
*****
Producto del sistema:
1) Realizar pedido
2) Ver estadísticas de ventas
3) Salir
*****
Elegí una opción: █
```

Qué pasa si elegimos la opción 1...

```
*****
TICKET - GO KITCHEN
*****
Producto del sistema:
1) Realizar pedido
2) Ver estadísticas de ventas
3) Salir
*****
Elegí una opción: █
```

Esta será la primera decisión que tendremos que tomar

```
Elegí una opción: 1

Restaurantes disponibles:
1) Burger House
2) Pizza Point
3) Sushi Go
Seleccione un restaurante: █
```

Si hipotéticamente elegimos la opción 2 nos saldrá la siguiente pantalla

```
Seleccione un restaurante: 2

Modalidad del pedido
1) Comer en el local
2) Para llevar
Seleccione una opción: █
```

Luego nos pondrán a elegir entre la variedad de comidas que ofrece el restaurante elegido

```
Menú de Pizza Point:
1) Pizza muzzarella - $6000
2) Empanadas - $800
Seleccione un producto: █
```

Posteriormente, y según la cantidad de unidades que deseemos llevarnos se aplicara un descuento del 20%. El descuento es si se llevan 3 o más productos.

```
Seleccione un producto: 2
Ingrese la cantidad: 5
¿Desea agregar toppings? s/n: s
```

```
Toppings disponibles:
1) Salsa picante - $200
Seleccione un topping: 1
¿Desea agregar otro topping? s/n: n
¿Desea agregar otro producto? s/n: n
```

Aquí se preguntará si le quiere agregar toppings y si quiere seguir comprando. Ambas se pueden responder por si o por no.

```
Resumen del pedido
Restaurante: Pizza Point
Modalidad: Comer en el local

Producto: Empanadas
Cantidad: 5
Toppings:
- Salsa picante: $200

Total sin descuento: $5000
Descuento aplicado: $1000.0
Total final: $4000.0
```

Y, Finalmente se mostrará el resumen del pedido.

Y si optamos por la segunda opción...

```
*****
TICKET - GO KITCHEN
*****
Producto del sistema:
1) Realizar pedido
2) Ver estadísticas de ventas
3) Salir
*****
Elegí una opción: █
```

La consola mostrara un pequeño resumen de las ventas discriminadas por restaurante.

```
*****
Elegí una opción: 2

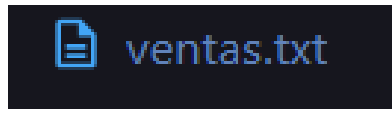
Estadísticas de ventas por restaurante

Restaurante: Burger House
Pedidos realizados: 0
Productos vendidos: 0
Total vendido: $0

Restaurante: Pizza Point
Pedidos realizados: 1
Productos vendidos: 5
Total vendido: $4000.0

Restaurante: Sushi Go
Pedidos realizados: 0
Productos vendidos: 0
Total vendido: $0
```

Además, el sistema genera un archivo de salida llamado “ventas.txt” con el siguiente formato:



```
Fecha: 29/06/2026 16:34:22
Restaurante: Pizza Point
Modalidad: Comer en el local
Productos vendidos: 2
Total vendido: $1600
-----
Fecha: 05/07/2026 14:39:18
Restaurante: Sushi Go
Modalidad: Comer en el local
Productos vendidos: 4
Descuento aplicado: $2400.0
Total vendido: $9600.0
-----
```

Y, por último, si queremos cerrar la aplicación escribimos un 3 por teclado...

```
*****
TICKET - GO KITCHEN
*****
Producto del sistema:
1) Realizar pedido
2) Ver estadísticas de ventas
3) Salir
*****
Elegí una opción: █
```

```
Elegí una opción: 3
Programa finalizado.
```

CONCLUSIÓN

A lo largo del desarrollo de este trabajo práctico se logró construir un sistema funcional de pedidos de comida llamado **“Go Kitchen”**, aplicando distintos conceptos vistos durante la materia. El programa permite seleccionar un restaurante, elegir productos, agregar toppings, definir la modalidad del pedido, calcular el total, aplicar descuentos y registrar las ventas realizadas.

Durante el proceso fue necesario organizar la información mediante distintas estructuras de datos, como listas y archivos. Se intentó utilizar archivos .txt como datos de entrada, pero no se logró concretar correctamente por dificultades en su funcionamiento. Por eso, el grupo decidió optar por el uso de listas de Python, una alternativa recomendada por la IA, para organizar restaurantes, productos, precios y toppings de manera más simple y funcional. También se utilizaron estructuras condicionales para validar opciones, aplicar descuentos y controlar el funcionamiento del menú principal. Las estructuras repetitivas permitieron recorrer listas, mostrar opciones al usuario y repetir acciones hasta recibir una entrada válida.

Otro aspecto importante fue la modularización del código mediante funciones. Esto permitió dividir el programa en partes más pequeñas y comprensibles, facilitando su lectura, corrección y mantenimiento. Además, se incorporó el uso de archivos de texto para guardar las ventas realizadas, evitando que la información anterior se pierda cada vez que se registra una nueva venta.